L12: The M/M/c Queue and Erlang-C

Simulation Without a Black Box

Outline

Learning Objectives

By the end of this lecture you will be able to:

- Write the M/M/c balance equations and state the stationary distribution.
- Compute the Erlang-C delay probability $C(c, \rho)$.
- Derive W_q from the Erlang-C formula and apply it to a worked example.
- Implement an M/M/c queue in SimPy with a single `Resource(capacity=c)`.
- Quantify the pooling gain: c pooled servers vs. c dedicated single-server queues.

M/M/c State Space and Balance Equations

With c parallel servers (FCFS, infinite buffer):

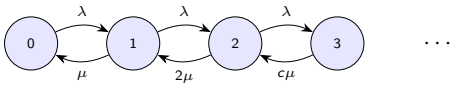
$$\rho = \frac{\lambda}{c\mu} < 1 \quad \text{(stability)}$$

Service rate in state n:

$$\mu_n = \begin{cases} n\mu & n < c \\ c\mu & n \geq c \end{cases}$$

Balance: $\lambda \pi_n = \mu_{n+1} \pi_{n+1}$
Stationary distribution:

$$\pi_n = \begin{cases} \dfrac{(c\rho)^n}{n!}\pi_0 & n \leq c \\ \dfrac{c^c \rho^n}{c!}\pi_0 & n > c \end{cases}$$



$c = 2$ servers
(state 2 onward: both busy)

Erlang-C Formula $C(c, \rho)$

Definition

$C(c, \rho)$ = probability that an arriving customer must wait (all servers busy):

$$C(c,\rho) = \frac{\dfrac{(c\rho)^c}{c!}\dfrac{1}{1-\rho}}{\displaystyle\sum_{n=0}^{c-1}\frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!}\frac{1}{1-\rho}}$$

Mean queue wait:

$$\boxed{W_q = \frac{C(c,\rho)}{c\mu - \lambda} = \frac{C(c,\rho)}{c\mu(1-\rho)}}$$

- When $c = 1$: $C(1, \rho) = \rho$, so $W_q = \rho/(\mu - \lambda)$ — recovers M/M/1.
- $C(c, \rho)$ is easy to compute iteratively (Jagerman recursion).

M/M/2 Worked Example

Given: $\lambda = 1.8$ customers/min, $\mu = 1$ customer/min per server, $c = 2$.

$\rho = \lambda/(c\mu) = 1.8/2 = 0.9$ (stable, since $\rho < 1$).

$$C(2, 0.9) = \frac{(1.8)^2/2 \cdot 1/(1-0.9)}{1 + 1.8 + (1.8)^2/2 \cdot 1/(1-0.9)} = \frac{16.2}{19} \approx 0.853$$

$$W_q = \frac{0.853}{2 \times 1 \times (1-0.9)} = \frac{0.853}{0.2} \approx 4.26 \text{ min}$$

Compare with two M/M/1 queues (each $\lambda' = 0.9$, $\mu = 1$)

$$W_q^{\text{M/M/1}} = \frac{0.9}{\mu - \lambda'} = \frac{0.9}{0.1} = 9 \text{ min}$$

Pooling halves the wait: one shared M/M/2 is far better than two dedicated M/M/1.

SimPy: Resource(capacity=c)

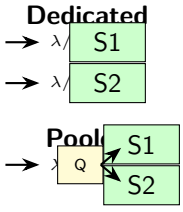
```python
import simpy, numpy as np

def customer(env, pool, lam, mu, rng, waits):
    arrive = env.now
    with pool.request() as req:
        yield req
        waits.append(env.now - arrive)
        yield env.timeout(rng.exponential(1/mu))

def source(env, pool, lam, mu, rng, waits):
    while True:
        yield env.timeout(rng.exponential(1/lam))
        env.process(customer(env, pool, lam, mu, rng, waits))

c   = 2
lam = 1.8
mu  = 1.0
rng = np.random.default_rng(42)
env = simpy.Environment()
pool = simpy.Resource(env, capacity=c)   # << only change vs M/M/1
waits = []
env.process(source(env, pool, lam, mu, rng, waits))
env.run(until=50_000)
print(f"Wq_sim={np.mean(waits):.3f}  Wq_theory=4.26")
```

Economies of Scale: Pooled vs. Dedicated Servers

Scenario: $\lambda = 1.8$, $\mu = 1$, 2 servers total.

Configuration	W_q	C
$2 \times$ M/M/1 (dedicated)	9.00 min	0.81
$1 \times$ M/M/2 (pooled)	4.26 min	0.85

Pooling *increases* $C(c, \rho)$ but *dramatically decreases* W_q because excess service capacity is never idle.



Optimal c: Given λ, μ, and a W_q Target

- Given target $W_q^* = 2$ min, $\lambda = 1.8$, $\mu = 1$:

c	$\rho = \lambda/(c\mu)$	$C(c, \rho)$	W_q
2	0.90	0.853	4.26
3	0.60	0.278	0.46
4	0.45	0.054	0.08

- $c = 2$ violates the target; $c = 3$ easily meets it.
- **Search strategy:** start at $\lceil \lambda/\mu \rceil$, increment c until $W_q \leq W_q^*$.
- For non-Markovian service, use simulation to verify the analytical estimate.

Key Takeaways

Core ideas from L12

1. The M/M/c stationary distribution has two regimes: $n < c$ (not all servers busy) and $n \geq c$ (queue forms).
2. **Erlang-C** $C(c, \rho)$ is the probability of waiting; $W_q = C(c, \rho)/(c\mu - \lambda)$.
3. SimPy needs only `Resource(env, capacity=c)` — no other code changes from M/M/1.
4. **Pooling wins:** one shared c-server queue always outperforms c dedicated single-server queues at the same load.
5. Find the minimum c that satisfies a W_q target by tabulating Erlang-C values.

Next: L13 — Service-time variability and the Pollaczek-Khinchine formula.